

Bipartite Augmenting Paths: From +1 Improvement to Alternating Reachability Sets

A detailed forward-derivation note, preserving the reasoning process

Writing principle

This is not a reverse-engineered textbook proof. It is organized as a forward derivation: problem pressure -> brute-force ideas -> failure modes -> state compression -> BFS form.

Contents

- 1. Why search for +1 instead of the maximum directly?
- 2. The +1 feasibility test and the FPT analogy
- 3. Brute-force augmenting paths and why they explode
- 4. Why edge sets are still too fine
- 5. The key turn: equivalence of path histories
- 6. Multi-source unmatched-left initialization
- 7. Right expansion, matched-edge return, and closure
- 8. The invariant that lets us forget history
- 9. Termination and correctness intuition
- 10. Complexity
- 11. What this derivation trains

1. Why search for +1 instead of the maximum directly?

Maximum matching is a global optimization problem: among all sets of pairwise vertex-disjoint edges, find one with maximum cardinality. If we attack the word "maximum" directly, the natural mental model quickly becomes combinatorial enumeration: choose this edge or not, rematch this vertex or not, replace which current matched edge, and so on.

A better first question is not "where is the maximum?" but: given a current matching M , what is the smallest positive improvement we can ask for? Since matching size is an integer, the smallest improvement is exactly +1.

Given the current matching M , is there a local modification that turns $|M|$ into $|M| + 1$?

This converts an optimization problem into a feasibility test. If a +1 modification exists, apply it. If no +1 modification exists, and we can prove that no such modification exists, then the current matching is already maximum.

Core idea

Maximum matching can be decomposed into repeated +1 feasibility tests. The +1 unit is not arbitrary; it is the smallest observable improvement in this discrete structure.

From this perspective, an augmenting path is not a definition pulled from a textbook. It is forced by the accounting of how to increase a matching by exactly one while preserving validity.

number of added edges - number of removed edges = 1
and the result is still a valid matching
=> matching size increases by 1

2. The +1 feasibility test and the FPT analogy

Your analogy to FPT is meaningful at the level of algorithmic design. In many fixed-parameter tractable problems, we do not directly solve the optimization problem first. Instead, we ask a parameterized feasibility question: is there a vertex cover of size at most k ? Is there a structure of size k ? Can the current target be achieved?

The matching process has a similar flavor: the current size is $|M|$, and we ask whether $|M|+1$ is feasible. If yes, we update M . If no, we stop. This reflects a broad pattern:

```
optimization problem
=> feasibility question
=> certificate search
=> state update if certificate exists
=> optimality proof if certificate does not exist
```

Strictly speaking, bipartite matching is not an FPT algorithm in the complexity-theoretic sense. FPT typically means a running time of the form $f(k) * \text{poly}(n)$, where the exponential part is isolated in a parameter k . The augmenting-path algorithm is stronger: it compresses the certificate search itself into ordinary polynomial-time reachability.

Aspect	Typical FPT thinking	Bipartite augmenting-path thinking
Question form	For parameter k , does a size- k structure exist?	Given M , does a +1 improvement exist?
Certificate	A bounded-size combinatorial object	An alternating path whose flip increases $ M $ by 1
Explosion source	Branching over parameter-dependent choices	Enumerating simple alternating paths
Breakthrough	Branching rules, kernels, parameter states	Equivalence of path histories into reachability states
Final complexity	$f(k) * \text{poly}(n)$	Polynomial time without a fixed parameter

So the precise statement is: the +1 idea shares the feasibility-certificate flavor of FPT, but the reason the matching algorithm is efficient is not a small parameter. It is the existence of a strong structural compression from paths to alternating reachability states.

3. Deriving augmenting paths from +1 accounting

Assume the current matching is M . We want a modification that keeps the result a valid matching and increases its size by one. A natural local operation is to flip edge status along a path: non-matching edges become matching edges; matching edges become non-matching edges.

For the flip to increase the cardinality by one, the path must contain one more non-matching edge than matching edge. This happens exactly when the path starts and ends with non-matching edges and alternates between non-matching and matching edges.

```
non-matching edges = matching edges + 1
flip the path
=> added edges exceed removed edges by 1
```

The endpoints must be unmatched: an unmatched left endpoint supplies the initial free position, and an unmatched right endpoint terminates the chain of displacement. Internal vertices are matched because they transmit the displacement through their unique current matched edge.

Position	Status	Role
Start	Unmatched left vertex	Supplies the initial free slot
Internal right vertex	Matched	Returns through its unique matched edge
Internal left vertex	Matched	Continues through a non-matching edge
End	Unmatched right vertex	Terminates the displacement and gives net +1

4. First brute-force version: enumerate all simple alternating paths

The direct construction is simple: start from an unmatched left vertex, walk through a non-matching edge, then a matching edge, then a non-matching edge, and so on, until reaching an unmatched right vertex. To avoid cycles, require the path to be simple.

This is logically valid but computationally disastrous. At each matched-left vertex, there may be many outgoing non-matching edges. Each chosen matched-right vertex returns uniquely to a matched-left partner, and then the branching repeats. Forbidding repeated vertices prevents infinite loops, but it does not prevent exponentially many simple paths.

Brute-force object: all simple alternating paths

Cycle prevention: no repeated vertex inside one path

Failure: the number of simple alternating paths can still be exponential

Main failure

A path is too detailed as a search object. It stores full history, but much of that history may be irrelevant for future expansion.

5. Failed compressions: edge subsets, matched-edge subsets, and last-edge anchors

A tempting next step is to compress the path into the set of matched edges it uses. After all, the flip removes exactly those matched edges and adds some non-matching edges. But this loses order and connectivity. The selected matched edges must be connectable by non-matching edges in the correct alternating order. If we store that order, we are back to path enumeration.

Another tempting idea is to enumerate the last edge of the augmenting path. The last edge must be a non-matching edge into an unmatched right vertex. Once its left endpoint is fixed, the previous matched edge is uniquely determined. This is useful as an observation, but it does not solve the prefix explosion: the third edge from the end is again a non-matching edge and can branch heavily.

Attempt	Why it looks useful	Why it fails
---------	---------------------	--------------

Enumerate paths	Keeps full legality	Exponential number of simple paths
Enumerate matched-edge subset	Coarser than path order	Loses connectivity and direction
Fix the final non-matching edge	Captures the end condition	The prefix still branches
DFS without backtracking	Uses uniqueness of matched edges	Non-matching edges still branch; wrong choice can miss a path

6. The key turn: do not choose a path; construct reachable vertex sets

The counterintuitive move is this: when branching explodes, do not try to choose the best branch. Instead, merge all branches that lead to the same future state. This is the central state-compression idea.

If two different alternating prefixes reach the same left vertex and in both cases the next legal edge type is non-matching, then their future expansion possibilities are identical. The exact prefix history is irrelevant.

Different histories:

L0 - R1 - L1 - R3 - L3

L4 - R2 - L1 - R3 - L3

If both place L3 in the same state:

"L3 is reachable and the next edge should be non-matching",
then future search only needs to remember L3.

Equivalence relation

Two path histories can be merged if they reach the same vertex with the same next-edge requirement. The future search capability is then the same.

This changes the object of search from paths to states. The state is no longer a full path. It is whether a vertex has been reached in the correct alternating layer.

7. Initial set: all unmatched left vertices

The search should begin with all unmatched left vertices. These are the only valid sources of an augmenting path from the left side, because they provide the initial free slot.

```
L_reached = { all currently unmatched left vertices }
R_reached = empty
```

This is a multi-source BFS. It is not merely a coding convenience. It is a state merge: if different unmatched-left sources eventually reach the same later left vertex, the future does not need to know which source was used.

The first expansion from these left vertices goes through graph edges. Since the left endpoints are unmatched, the first edge cannot be a current matching edge. If this expansion reaches an unmatched right vertex, a length-1 augmenting path has already been found.

8. Expansion rule: non-matching edges to the right, matching edges back to the left

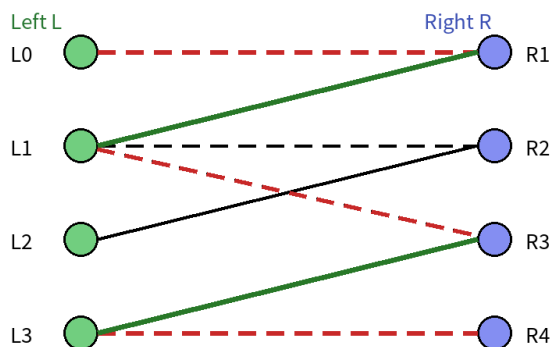
From a set of reached left vertices, expand through non-matching edges to right vertices. If any reached right vertex is unmatched, stop: an augmenting path exists. If a reached right vertex is matched, use its unique matching edge to return to the corresponding left vertex, and add that left vertex to the reached-left set.

Current state: a set of reached left vertices.

1. Expand along non-matching edges to right vertices.
2. If an unmatched right vertex is reached, an augmenting path is found.
3. If a reached right vertex is matched, follow its unique matched edge back to the left side.
4. Add newly reached left vertices and continue.

The matching edges are what make compression possible: a matched right vertex has exactly one matched left partner. The non-matching edges create branching, but after branching we merge by vertex reachability instead of preserving separate path histories.

Alternating reachability: propagate state sets, not paths



Legend

solid = matching edge
dashed = non-match edge
red/green = one augmenting path

State-set view

Reached left nodes are enough;
the full path history is forgotten.
A matched right node uniquely
maps to its matched left partner.

9. The invariant that lets us forget path history

The correctness of the compression rests on an invariant:

Every vertex in L_{reached} is the endpoint of some alternating prefix starting from an unmatched left vertex, and from it the next legal edge should be non-matching.

Every vertex in R_{reached} is reached from L_{reached} through a non-matching edge.

If it is unmatched, the prefix is already an augmenting path.

If it is matched, the matched edge uniquely returns to the left side.

This invariant explains why vertex sets are enough. Once a left vertex is reached in this state, future expansions depend only on that vertex and the current matching, not on the particular prefix that reached it.

The handle

The handle is not BFS by itself. The handle is the proof that alternating path history can be collapsed into reached vertices in alternating layers.

10. Termination and correctness intuition

Each expansion either discovers an unmatched right vertex or adds at least one new vertex to the reached sets. If no new vertex can be added, the alternating reachability closure is stable.

If the closure stabilizes without reaching any unmatched right vertex, then no alternating path from any unmatched left vertex to any unmatched right vertex exists. Therefore no augmenting path exists. By Berge's theorem, a matching is maximum if and only if it has no augmenting path.

If an unmatched right vertex is reached:

flip the corresponding alternating path and increase $|M|$ by 1.

If the closure stabilizes without reaching one:

no augmenting path exists; the current matching is maximum.

11. Why this is not DFS without backtracking

Matched edges are unique only in one direction of the alternating process: from a matched right vertex back to its matched left partner. The branching remains in the other direction: from a reached left vertex to many possible non-matching right neighbors. A DFS that does not backtrack can choose a dead branch and miss an augmenting path through another branch.

The set-based BFS avoids this by not committing to one branch. It admits all possible right neighbors, then merges the resulting states. This is not merely changing search order; it changes the representation of the search space.

12. Complexity: from exponential paths to polynomial states

Brute-force path enumeration is exponential in the number of simple alternating paths. The reachability-set version only adds each vertex to a reached set a bounded number of times, and each edge is inspected a bounded number of times in a standard implementation. Thus one augmenting-path search is polynomial, typically $O(E)$ for the BFS-style reachability pass.

Repeating this search and flipping one augmenting path at a time yields a polynomial algorithm. Hopcroft-Karp further improves the bound by finding multiple shortest augmenting paths per phase, but the essential discovery here is already the compression from path histories to reachability states.

Version	Search object	Complexity intuition
Brute-force paths	All simple alternating paths	Exponential
Edge/subset compression	Matched-edge subsets or edge histories	Still can be exponential

Reachability sets	Alternating-layer vertex states	Polynomial
-------------------	---------------------------------	------------

13. What this derivation trains

The main training value is not memorizing bipartite matching. It is the algorithm-discovery pattern:

- Turn a global optimization problem into the smallest possible improvement test.
- Define the certificate that would prove improvement.
- Allow a brute-force version first, so the exploding object is explicit.
- Locate exactly where the exponential branching comes from.
- Search for an equivalence relation on histories.
- Use an invariant to justify forgetting history.
- Convert certificate enumeration into closure construction.

This is why the reasoning felt similar to FPT or NP-hard problem exploration. In both settings, one often asks a feasibility question and then fights the explosion of certificates. Here the special structure of matching is strong enough to compress the entire search into polynomial-time alternating reachability.

Final summary

The visible algorithm is BFS. The hidden idea is: +1 certificate + failure of path enumeration + equivalence of histories + alternating reachability closure.

14. One-sentence algorithm

Start from all unmatched left vertices.
 Repeatedly expand from reached left vertices along non-matching edges to right vertices.
 If an unmatched right vertex is reached, recover and flip the augmenting path.
 If a reached right vertex is matched, follow its unique matched edge back to the left side.
 If no new vertices can be reached, no augmenting path exists.

This sentence is the algorithm. The derivation above explains why it is not just "doing BFS", but BFS after the exponential path histories have been collapsed into polynomially many alternating reachability states.